

# Cost-effective Vehicle Recognition System in Challenging Environment Empowered by Micro-Pulse LiDAR and Edge AI

1<sup>st</sup> Junyue Jiang  
*Civil and System Engineering  
Johns Hopkins University  
Baltimore, MD  
jjjiang67@jh.edu*

2<sup>nd</sup> Hongliang Lu  
*The Hong Kong University of  
Science and Technology (Guangzhou)  
Guangzhou, China  
hlu592@connect.hkust-gz.edu.cn*

3<sup>rd</sup> Chenxi Liu  
*Civil & Environmental Engineering  
University of Washington  
Seattle, USA  
lcx2017@uw.edu*

4<sup>th</sup> Meixin Zhu  
*The Hong Kong University of  
Science and Technology (Guangzhou)  
Guangzhou, China  
meixin@ust.hk*

5<sup>th</sup> Yiran Chen  
*Electrical and Computer Engineering  
Duke University  
Durham, USA  
yiran.chen@duke.edu*

6<sup>th</sup> Hao Frank Yang\*  
*Civil and System Engineering  
Johns Hopkins University  
Baltimore, MD  
haofrankyang@jhu.edu  
\*Corresponding author*

**Abstract**—Vehicle recognition and classification are critical for a number of traffic applications, e.g., traffic signal control, traffic flow modeling, tolling, and logistics optimization. Commonly used sensing systems are mainly counted on in-pavement loops or surveillance video cameras, while both of them have their inherent limitations. Leveraging micro high-speed pulse LiDAR mounted overhead of travel lanes, this study proposes Compact LiDAR Empowered Vehicle Enhancing-minority Recognition (CLEVER) system, a real-time cost-effective vehicle detection and classification framework that is empowered by edge Artificial Intelligence (AI). Based on the customized minority-enhancing vehicle classification deep neural network, the CLEVER system outperforms cutting-edge LiDAR-based vehicle classification methods up to 15.98% true-positive rate in classifying ten types of vehicles. Furthermore, by highly integrating the hardware, the pre-processing algorithm and the classification neural network into an edge computing node, the CLEVER system only consumes 10% of the cost in LiDAR systems and works perfectly in a plug-and-play mode with a negligible sub-second inference time (212ms to 459ms). The proposed CLEVER system offers an affordable end-to-end solution that can benefit traffic operators by collecting more accurate and reliable vehicle classification data streams and that can lead to a more efficient and flexible ITS.

**Index Terms**—Vehicle classification; deep learning; Light Detection and Ranging (LiDAR); edge computing; data augmentation.

Vehicle detection and recognition are essential in intelligent transportation systems for traffic control, congestion prediction, and logistics. Common technologies include loop detectors [1], video-based systems [2], [3], [4], and others [5], [6]. However, loop detectors suffer from accuracy and installation issues [7], [8], while video systems, though enhanced by deep learning, face high costs and performance limitations under difficult conditions [9], [10].

Researchers are exploring alternatives like wireless sensors

[11], radar, and particularly LiDAR [12], known for its precision and reliability in low-light via stimulated light or laser waves [13], [14], [15]. However, LiDAR integration presents challenges:

- **Cost vs. Performance:** High-precision features require costly setups like rotating scanners, complicating widespread use [16].
- **Minority Class Recognition:** Existing methods, relying on outdated algorithms, often neglect minority vehicle classes [17], [18].
- **Signal Outliers from Reflective Surfaces:** LiDAR struggles with data reliability on surfaces like glass, necessitating advanced correction methods [19].

The need for advanced technologies is highlighted by previously identified gaps, prioritizing the development of a compact, cost-effective, and durable LiDAR sensor. The emerging modulated pulse LiDAR, operating up to 10kHz, provides high precision without the extensive data typical of conventional LiDAR systems, which often only adds environmental noise irrelevant to vehicle classification [16], [20], [21]. Instead of comprehensive point clouds, capturing just the vehicle's contour may suffice.

**State-of-the-art deep learning models**, utilizing advanced neural network techniques, enhance our ability to classify vehicles accurately by focusing on critical features, even within imbalanced datasets. These methods have significantly shaped our minority-enhancing deep neural network, part of the new Compact LiDAR Empowered Vehicle Enhancing-minority Recognition (CLEVER) system.

This pioneering system integrates high-speed compact pulse LiDAR with an edge computing node, achieving remarkable classification accuracy across ten vehicle types with

substantial operational efficiencies:

- A real-time, plug-and-play framework significantly reduces size, weight, cost, and complexity.
- Custom pre-processing workflows improve LiDAR data reliability for vehicle contour representation.
- Our vehicle classification strategy effectively balances identity and environmental data, improving the true positive rate (TPR) by up to 15.98%.

The article’s structure is compartmentalized into six segments. Section II demystifies the CLEVER edge hardware ecosystem, while Section III delves into the intricacies of our algorithmic workflow. Sections IV and V spotlight the CLEVER Vehicle Classification (CLEVER VC) neural network blueprint and its empirical evaluations, situating it against other avant-garde contenders. The concluding remarks and prospective avenues for exploration are reserved for the final section.

## I. CLEVER EDGE SYSTEM DESCRIPTION

### A. High-speed Compact (Micro) Pulse LiDAR Description

“Time-of-flight” (ToF) measurement by pulsed laser is the fundamental concept for single beam high-speed compact pulse LiDAR. The basic principle is shown in Fig. ??(a). Compared with other scanning or flash LiDAR sensor (i.e., VLP-16 LiDAR) mentioned earlier, the high-speed compact pulse LiDAR has a relatively limited field of view. Despite this, the advantages are still obvious and can be summarized as follows:

- **Cost-effective.** The high-speed pulse LiDAR only costs 3% to 10% of the prices for the scanning LiDARs used in previous research.
- **High-speed.** The pulse measuring frequency can reach up to 5k-10kHz, which is hundreds or even thousands of times higher than scanning LiDAR.
- **High-reliability** Controlling only one or several beams, the LiDAR control algorithm can change dynamically in response to the external environment more quickly and then adjust the parameters of the laser beam to obtain a reliable detection result (especially in rain and snow conditions).
- **Lower Data Volume** The 3D data volume obtained by scanning LiDAR is huge (20-50 MB/s) and requires complicated processing and storage procedures. High-speed Compact Pulse LiDAR only has 2D sequence information, which is light-weight and friendly to data processing and storage terminals.
- **Lighter and Smaller** Since complex mechanical rotating units are no longer needed, the size and weight of the sensor can be greatly reduced.

### B. Hardware System

The hardware system of the CLEVER project is built around two principal components: an edge computing node and a compact pulse LiDAR. We utilize the Jetson Xavier NX [22] as the embedded edge platform, equipped with a 6-core ARM CPU, 384 CUDA cores, and 48 tensor cores GPU, optimized for real-time data processing from the

LiDAR sensor. This setup enables efficient data handling and computation, leveraging the full computing capabilities of the edge node in a synchronized workflow.

Additionally, the Benewake TF03 single beam pulse LiDAR is integrated within this system. It operates on the Pulse Time of Flight (PTOF) principle for accurate range measurement, seamlessly integrating with the Jetson Xavier NX to form a robust hardware platform for the CLEVER system.

### C. Implementation Illustration

One CLEVER edge node can contain up to four TF03 LiDAR units to cover four separate travel lanes. The sensor can be deployed on the traffic mast arm, L-shape arm, overpass, or even signal light pole in an overhead configuration. The height is adjustable from two meters to eighty meters above the targets. Each LiDAR unit emits a single laser beam perpendicular to traffic on the XY plane and measures distances between the sensor and vehicles passing in front of the sensor continuously with a frequency from 100Hz to 10kHz measurements per second. The raw sensor readings (including distance and strength) and timestamp of each measurement are recorded.

## II. ALGORITHMS WORKFLOW

### A. System Input

As depicted in Fig. 1, the CLEVER system’s algorithmic workflow involves LiDAR data input, attribute input, data pre-processing, and the classification neural network. Inputs include LiDAR measurements (distance and strength) and environmental and object-level attributes such as speed limits, time of day, and object contour dimensions. These inputs are vital for the subsequent data cleansing and pre-processing stages, aimed at enhancing minority vehicle classification.

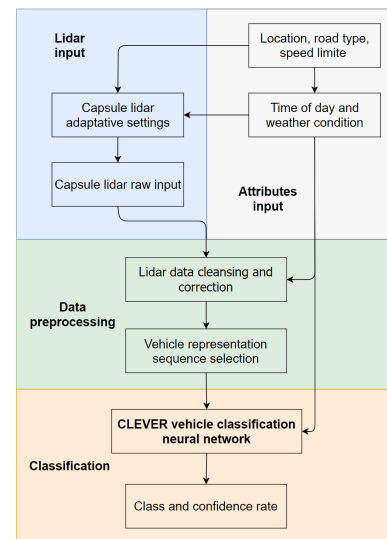


Fig. 1. The algorithm workflow of CLEVER, including LiDAR and attributes input, data pre-processing component and classification neural network.

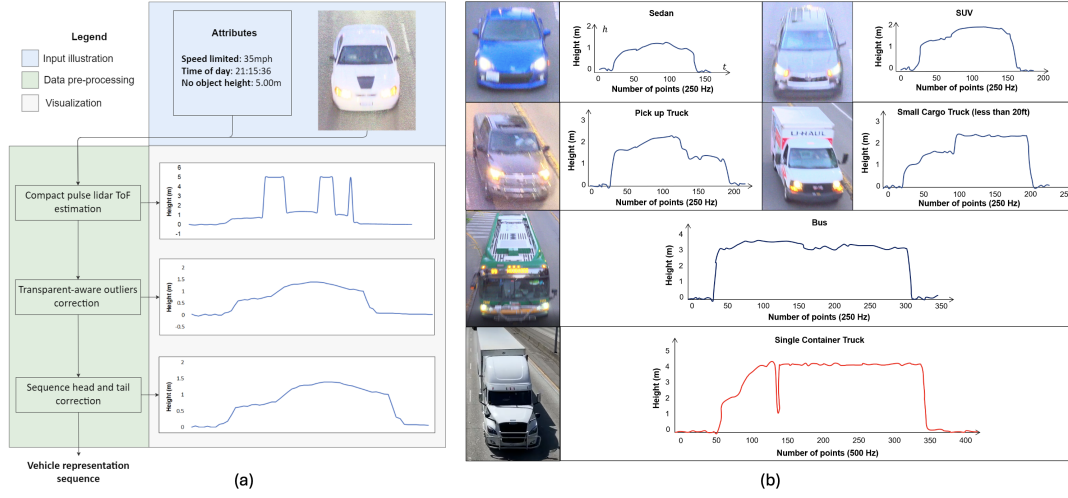


Fig. 2. (a) Illustration of data pre-processing procedures of CLEVER edge nodes. The top figure is the belonged white sedan captured by a video recorder. (b) Vehicles representation sequences examples. The dark blue lines are collected in the urban area and the frequency is 250Hz. The single container truck sequence (red) is collected at the freeway using 500Hz frequency. Left figures are the belonged vehicles captured by a video recorder.

### B. Data Cleansing and Pre-processing

Data cleansing ensures the quality of LiDAR features and attribute information. The LiDAR system's inherent characteristics might lead to poor data from highly transmissive surfaces like glass. In such cases, the data is corrected using a shape-aware correction workflow based on the actual scenario (Fig. 2(a)). Corrected data forms vehicle representation sequences, defined by critical points:

- **Vehicle Starting Point ( $p_0^i$ ):** Identified when five continuous detection records above 30cm signify a vehicle head.
- **Vehicle Representation Sequence Starting Point ( $s_0^i$ ):** Begins from  $p_0^i$ , covering vehicle features and background conditions.
- **Vehicle Ending Point ( $p_{max}^i$ ):** Defined by five continuous detection records below 20cm, marking the end of the vehicle.
- **Vehicle Representation Sequence Ending Point ( $s_{max}^i$ ):** Extends from  $p_{max}^i$ , ensuring complete vehicle capture.

These reference points ensure a precise and comprehensive capture of vehicle contours for input into the CLEVER-VC neural network, as illustrated in Fig. 2(b).

### III. CLEVER VEHICLE CLASSIFICATION NEURAL NETWORK

To effectively classify vehicle types by their contours, we propose the CLEVER vehicle classification (CLEVER VC) neural network, designed for sequential data recognition. Drawing inspiration from text classification models in natural language processing (NLP) [23], [24], the model adeptly learns and integrates both temporal dependency and spatial distribution features. Additionally, it considers various environmental and identity-level attributes such as time of day, vehicle contour estimation deviation, sensor installation location, and speed limit. These attributes are incorporated

through a learnable component that enhances the model's ability to process spatial and temporal data. **In essence, CLEVER VC utilizes both the vehicle representation sequence ( $S_i$ ) and an attributes sequence ( $A_i$ ) as inputs to achieve precise classification.**

#### A. Attributes Embedding

The CLEVER VC neural network incorporates both environmental and object-specific attribute features into the classification process. Environmental features include road speed limit (30mph/60mph) and time of day; object-specific features cover aspects such as contour sequence length, data recording frequency (250Hz/500Hz), object height, and passing duration. Due to the complexity and interdependencies of these attributes, a dedicated embedding component is integrated within the network to manage them [25], [26].

This embedding component transforms attribute data into a learnable vector space, enhancing feature representation and differentiation [27], [28]. The transformation is mathematically defined as:

$$Att_{a_j^i} = W^{a_j^i} \cdot a_j^i \quad (1)$$

where  $a^i$  represents the attribute data for sequence  $i$ ,  $j$  indexes within the sequence, and  $W^{a^i}$  denotes the learnable weight matrix that maps input attributes from their original space  $A$  to an embedding space  $E^{a^i}$ . Each attribute is mapped to a suitable vector space to optimize feature recognition.

After embedding, the output vectors  $E^{a^i}$  are concatenated with the vehicle contour sequence  $S_i$  to form a combined input  $SE_i$  for the subsequent convolutional block as in Fig. 3.

#### B. Convolutional Block

The convolutional block in our model employs temporal convolutions, also known as 1D convolutions, which are effective for time series sequence classification [29], [30]. Our architecture uses a Fully Convolutional Network (FCN)

[5], [31] that functions primarily as a feature extractor. This block works in conjunction with an attention LSTM block to enhance the capture of sequential features.

The convolutional process is defined by:

$$C_{SE_i^j}^{conv} = \sigma_{cnn}(W_{conv} * SE_i^{j:j+k-1} + \varepsilon) \quad (2)$$

where  $C_{SE_i^j}^{conv}$  is the output from the  $j^{th}$  dimension of the convolutional layer,  $W_{conv}$  is the trainable weight matrix,  $\sigma_{cnn}$  is the activation function,  $k$  is the kernel size, and  $\varepsilon$  represents the bias term.

The convolutional block includes three stacked temporal convolution layers with filter sizes of 128, 256, and 128, designed similarly to the blocks in the CNN architecture outlined by [23]. Each layer is equipped with batch normalization and a Rectified Linear Unit (ReLU) activation function. Post-processing involves global average pooling to reduce parameter count and simplify the model before classification, averaging out each channel's outputs.

### C. Attention LSTM Block

The LSTM module is integrated into our model to address temporal dependencies in input sequences. Each LSTM neuron comprises three gates: the input gate  $i_{(t)}$ , the output gate  $o_{(t)}$ , and the forget gate  $f_{(t)}$ . These gates manage memory updates and output generation, controlling the flow of information through weights  $w^{(t)}$  and previous outputs  $h_{(t-1)}$ . The memory process involves creating new memory  $\tilde{c}_{(t)}$  and updating the final memory cell  $c_{(t)}$ , with the new state  $h_{(t)}$  determined by the output gate.

Recognizing that different regions of a vehicle sequence may contribute variably to classification, we employ an attention mechanism to highlight key features. This mechanism assigns a learned weight  $\alpha_i$  to each output in the sequence  $h_i$ , emphasizing important characteristics such as the vehicle's backside or the length of a container. These weights enhance the model's ability to distinguish between different vehicle types, like sedans and SUVs, or different truck sizes. Our model uses an attention-aware LSTM layer to dynamically adjust the significance of each sequence part based on these weights, optimizing classification accuracy.

$$\mathcal{F}_{P_j} = \sum_{i=1}^{L_{h_i}} \alpha_i * h_i \quad (3)$$

$$\alpha_i = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{i,k})} \quad (4)$$

In (10), the  $e_{ij} = fa(s_{i-1}, h_j)$  is an alignment model, the output scores how well the input around  $j^{th}$  record and the output at position  $i_{th}$  aligned. In this research, we use the alignment mechanism proposed by [32]. The integrated model directly computes a soft alignment, which allows the gradient of the cost function to be backpropagated.

After the two blocks obtain the features, a concatenate layer is used to link both parts' features together, and then using a fully connected layer to transfer the original dimension of features to the number of classes. Finally, by

passing a softmax operation layer, the probability of different classes can be obtained.

### D. Loss Function

The CLEVER VC system, tailored for sequential classification, utilizes cross-entropy loss as detailed in [33]. This function, ideal for assessing discrepancies between predicted class probabilities and actual labels, is expressed mathematically as:

$$\mathcal{L}_{Xent} = -\frac{1}{N} \sum_{j=1}^N y_j \log(\hat{y}_j) \quad (5)$$

Here,  $\mathcal{L}_{Xent}$  signifies the cross-entropy loss,  $N$  the number of classes,  $y_j$  the one-hot encoded true class, and  $\hat{y}_j$  the predicted probability of each class.

Cross-entropy is pivotal for optimizing the model during training by effectively minimizing the prediction error, enhancing classification accuracy. It also adeptly manages class imbalance by proportionally penalizing incorrect predictions, which is crucial for the system's effective vehicle classification.

## IV. EXPERIMENT

### A. Dataset Description

1) *Real-world Data Collection*: To evaluate the CLEVER system's capability comprehensively, data was collected in Seattle across various times, including both daytime and nighttime conditions. The LiDAR sensors were strategically installed on overpasses, covering an urban road with a 35 mph speed limit and a freeway with a 60 mph speed limit. A total of 909 vehicles were recorded during daylight, while 631 vehicles were captured at night. These vehicles were classified into ten categories: sedans, SUVs, minivans, pickup trucks, standard city buses, bi-articulated buses, small trucks (under 20ft), 20ft container trucks, 40ft container trucks, and multi-trailer trucks. To ensure the accuracy of the collected data, video recordings were also made as supplementary validation.

2) *Data Augmentation*: Beyond real-world data collection, the team enhanced the dataset through synthetic augmentation techniques. Using a Mask R-CNN [34] with a ResNeXt-101-FPN backbone [35], trained on the COCO dataset [36], masks of vehicles were generated. This model included 100 distinct vehicle types encompassing a variety of classes: 15 sedans, 15 SUVs, 15 minivans, 10 pickup trucks, 5 each of standard city and bi-articulated buses, 15 small trucks, 10 20ft container trucks, 5 40ft container trucks, and 5 multi-trailer trucks. Each vehicle's mask was scaled to actual dimensions to reflect true vehicle heights and create realistic vehicle contours from top-down LiDAR views.

To simulate varying traffic conditions and to robustly test the CLEVER system, sequence lengths based on real data distributions were statistically adjusted to represent different vehicle lengths and traffic densities. Vehicles were re-sampled 5 to 10 times, with sequence lengths modeled after a Gaussian distribution, adjusting for variations in speed and traffic

TABLE I  
THE TPR RESULTS COMPARISON OF CLEVER VC VERSUS BASELINES.

| Type                                                     | Class                   | Accuracy (TPR)     |                    |                    |                    |                             |
|----------------------------------------------------------|-------------------------|--------------------|--------------------|--------------------|--------------------|-----------------------------|
|                                                          |                         | SVM                | k-NN               | RF                 | FCN                | CLEVER VC                   |
| Without Data Augmentation<br>/<br>With Data Augmentation | Sedan                   | 74.51/77.27        | 75.49/78.79        | 78.43/80.30        | 88.24/88.64        | <b>95.10/91.67(↓ 3.43)</b>  |
|                                                          | SUV                     | 76.92/78.70        | 79.49/81.48        | 83.33/83.33        | 88.46/90.74        | <b>92.31/95.37(↑ 3.06)</b>  |
|                                                          | Minivan                 | 52.63/67.65        | 73.68/70.59        | 68.42/82.35        | 78.95/85.29        | <b>89.47/88.24(↓ 1.24)</b>  |
|                                                          | Pickup Truck            | 79.07/79.37        | 81.40/82.54        | 88.37/87.30        | 93.02/95.24        | <b>93.02/96.83(↑ 3.80)</b>  |
|                                                          | Standard city bus       | 64.29/75.86        | 71.43/72.41        | 71.43/75.56        | 78.57/86.21        | <b>85.71/89.66(↑ 3.94)</b>  |
|                                                          | Bi-articulated bus      | <b>66.67/66.67</b> | <b>66.67/75.00</b> | <b>66.67/75.00</b> | <b>66.67/83.33</b> | <b>66.67/83.33(↑ 16.67)</b> |
|                                                          | Small truck             | 72.73/69.23        | 72.73/73.08        | <b>81.82/76.92</b> | 63.64/76.92        | <b>72.73/88.46(↑ 15.73)</b> |
|                                                          | 20ft containers truck   | 77.27/73.81        | 71.73/71.43        | 77.03/78.57        | <b>86.36/85.71</b> | <b>81.82/88.10(↑ 6.28)</b>  |
|                                                          | 40ft containers truck   | 58.33/74.07        | 58.33/70.37        | 58.33/81.48        | 58.33/81.48        | <b>66.67/85.19(↑ 6.28)</b>  |
|                                                          | Multi-trailer truck     | <b>66.67/66.67</b> | <b>66.67/66.67</b> | <b>66.67/66.67</b> | <b>66.67/77.78</b> | <b>66.67/88.89(↑ 18.52)</b> |
|                                                          | <b>Overall Accuracy</b> | <b>73.65/75.73</b> | <b>76.25/77.18</b> | <b>79.17/81.12</b> | <b>85.33/87.97</b> | <b>90.20/91.70(↑ 1.5)</b>   |

volume. Augmented sequences also included non-vehicle LiDAR data to mimic environmental impacts, enhancing the training dataset's diversity and complexity.

### B. CLEVER VC Neural Network Performance Evaluation

1) *Baseline models*: This study benchmarked CLEVER-VC against five different baselines: three traditional machine learning approaches and one deep learning approach, including a linear classifier SVM [37], a statistical pattern recognition method k-NN [38], an ensemble learning method RF [37], and a CNN-variant method FCN [39].

2) *Implementation and Comparison*: Key parameters for the model comparison and implementation include:

- **Data Split**: The dataset comprises 2401 vehicles, split into 70% training (1681 vehicles), 10% validation (240 vehicles), and 20% testing (480 vehicles). A subset of 1541 real vehicles was also used to demonstrate the effectiveness of vision-augmented data.
- **LSTM Neurons**: The model employs 64 LSTM neurons.
- **Learning Rate and Optimizer**: RMSprop is used with an initial learning rate of  $1e-3$ , reduced by  $1/\sqrt[3]{2}$  every 50 epochs.
- **Edge Node Deployment**: The model is deployed on a Jetson Xavier NX with PyTorch v1.4.0, handling sequence generation, pre-processing, and inference in a plug-and-play manner on edge nodes.

3) *Implementation and Comparison*: Key parameters for our model comparison and implementation include:

The detailed model comparison is presented in Table I. Deep learning models outperformed traditional approaches both on class-level accuracies and on the overall accuracy. Based on experiment result, SVM always shows the worst performance; RF is the best traditional machine-learning model and shows better precision than k-NN and SVM. The same situation also can be found from the literature [16]. Furthermore, deep learning is proved to be a promising and encouraging approach in this classification task. The FCN is better than RF by 6% to 7%. The CLEVER VC shows

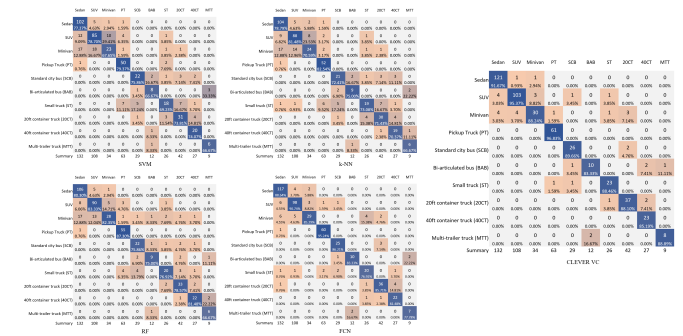


Fig. 3. The classification confusion matrix of the baseline models and CLEVER VC.

the best performance in all classes, with the best accuracy on the sedan with 95.10% (no augmentation) and 96.83% (with augmentation). Furthermore, the comparison clearly shows all methods failed at the minority classes, such as buses and 40 ft container trucks without data augmentation. However, by including visual augmented data into the training and evaluation process, the performance on minority classes significantly improved, with a maximum 18.52% gain of accuracy on the multi-trailer truck. The customized data augmentation process successfully boosts the performances of minority-class and also lead to a 1.5% accuracy gain for overall accuracy. The detailed accuracy of each class, with or without data augmentation procedure, can be found in Table I.

### V. CONCLUSIONS AND FUTURE WORK

This paper presents a real-time plug-and-play vehicle counting and classification framework based on compact pulse LiDAR and edge artificial intelligence. A comprehensive LiDAR pre-processing workflow is proposed, automatically correcting the outlying values and providing reliable inputs for the classification algorithm. A customized minority-class vehicle classification methodology with environmental attribution integration component is customized and fully integrated into the CLEVER system and achieved significantly leading TPR to other advanced methodologies.



Based on the benchmarking results, the CLEVER system has several advantages over previous solutions, including only about 10% of sensor cost, online data processing, and classifications, etc. Through the edge empowered end-to-end skeleton, the vehicle counting and classification algorithms can be finished in an online real-time manner. To build a more powerful and reliable vehicle classification system, future research will focus on improving algorithm accuracy, especially how to solve the vehicle recognition task under unbalanced data distributions.

## REFERENCES

- [1] H. Rakha and W. Zhang, "Estimating traffic stream space mean speed and reliability from dual-and single-loop detectors," *Transportation Research Record*, vol. 1925, no. 1, pp. 38–47, 2005.
- [2] A. Gholamhosseinian and J. Seitz, "Vehicle classification in intelligent transport systems: An overview, methods and software perspective," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 2, pp. 173–194, 2021.
- [3] H. Yang, J. Cai, M. Zhu, C. Liu, and Y. Wang, "Traffic-informed multi-camera sensing (tims) system based on vehicle re-identification," *IEEE Transactions on Intelligent Transportation Systems*, 2022.
- [4] H. F. Yang, J. Cai, C. Liu, R. Ke, and Y. Wang, "Cooperative multi-camera vehicle tracking and traffic surveillance with edge artificial intelligence and representation learning," *Transportation research part C: emerging technologies*, vol. 148, p. 103982, 2023.
- [5] J. Dai, Y. Li, K. He, and J. Sun, "R-fcn: Object detection via region-based fully convolutional networks," in *Advances in neural information processing systems*, pp. 379–387, 2016.
- [6] Y. Li, A. Y. C. Tok, Z. Sun, S. G. Ritchie, and K. R. Allu, "Lidar vehicle point cloud reconstruction framework for axle-based classification," *IEEE Sensors Journal*, vol. 23, pp. 11168–11180, June 2023.
- [7] D. R. Middleton, R. Parker, and R. Longmire, "Investigation of vehicle detector performance and atms interface," tech. rep., Texas Transportation Institute, Texas A & M University System College ..., 2007.
- [8] S. R. E. Datondji, Y. Dupuis, P. Subirats, and P. Vasseur, "A survey of vision-based traffic monitoring of road intersections," *IEEE transactions on intelligent transportation systems*, vol. 17, no. 10, pp. 2681–2698, 2016.
- [9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 779–788, 2016.
- [10] G. Zhang, R. P. Avery, and Y. Wang, "Video-based vehicle detection and classification system for real-time traffic data collection using uncalibrated video cameras," *Transportation research record*, vol. 1993, no. 1, pp. 138–147, 2007.
- [11] M. Bottero, B. Dalla Chiara, and F. P. Deflorio, "Wireless sensor networks for traffic monitoring in a logistic centre," *Transportation Research Part C: Emerging Technologies*, vol. 26, pp. 99–124, 2013.
- [12] W. Liu, W. Sun, and Y. Liu, "Dloam: Real-time and robust lidar slam system based on cnn in dynamic urban environments," *IEEE Open Journal of Intelligent Transportation Systems*, 2021.
- [13] Z. Meng, X. Xia, R. Xu, W. Liu, and J. Ma, "Hydro-3d: Hybrid object detection and tracking for cooperative perception using 3d lidar," *IEEE Transactions on Intelligent Vehicles*, 2023.
- [14] J. Cai, Y. Wang, and J.-N. Hwang, "Ace: Ally complementary experts for solving long-tailed recognition in one-shot," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 112–121, 2021.
- [15] A. Jazayeri, H. Cai, J. Y. Zheng, and M. Tuceryan, "Vehicle detection and tracking in car video based on motion model," *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 2, pp. 583–595, 2011.
- [16] J. Wu, H. Xu, Y. Zheng, Y. Zhang, B. Lv, and Z. Tian, "Automatic vehicle classification using roadside lidar data," *Transportation Research Record*, vol. 2673, no. 6, pp. 153–164, 2019.
- [17] J. Zhao, H. Xu, H. Liu, J. Wu, Y. Zheng, and D. Wu, "Detection and tracking of pedestrians and vehicles using roadside lidar sensors," *Transportation research part C: emerging technologies*, vol. 100, pp. 68–87, 2019.
- [18] O. Sahin, R. V. Nezafat, and M. Cetin, "Methods for classification of truck trailers using side-fire light detection and ranging (lidar) data," *Journal of Intelligent Transportation Systems*, pp. 1–13, 2020.
- [19] P. Foster, Z. Sun, J. J. Park, and B. Kuipers, "Visagge: Visible angle grid for glass environments," in *2013 IEEE International Conference on Robotics and Automation*, pp. 2213–2220, IEEE, 2013.
- [20] C. Liu, H. Yang, M. Zhu, F. Wang, T. Vaa, and Y. Wang, "Real-time multi-task environmental perception system for traffic safety empowered by edge artificial intelligence," *IEEE Transactions on Intelligent Transportation Systems*, 2023.
- [21] B. Schlager, T. Goelles, M. Behmer, S. Muckenhuber, J. Payer, and D. Watzenig, "Automotive lidar and vibration: Resonance, inertial measurement unit, and effects on the point cloud," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 426–434, 2022.
- [22] "Nvidia jetson xavier series." <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-xavier-series/>. Accessed: 23-April-2024.
- [23] F. Karim, S. Majumdar, H. Darabi, and S. Chen, "Lstm fully convolutional networks for time series classification," *IEEE access*, vol. 6, pp. 1662–1669, 2017.
- [24] F. Karim, S. Majumdar, and H. Darabi, "Insights into lstm fully convolutional networks for time series classification," *IEEE Access*, vol. 7, pp. 67718–67725, 2019.
- [25] W. Zheng, H. F. Yang, J. Cai, P. Wang, X. Jiang, S. S. Du, Y. Wang, and Z. Wang, "Integrating the traffic science with representation learning for city-wide network congestion prediction," *Information Fusion*, vol. 99, p. 101837, 2023.
- [26] H. Yang, C. Liu, Y. Zhuang, W. Sun, K. Murthy, Z. Pu, and Y. Wang, "Truck parking pattern aggregation and availability prediction by deep learning," *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [27] K. Kowsari, K. Jafari Meimandi, M. Heidarysafa, S. Mendu, L. Barnes, and D. Brown, "Text classification algorithms: A survey," *Information*, vol. 10, no. 4, p. 150, 2019.
- [28] H. Yang, C. Liu, M. Zhu, X. Ban, and Y. Wang, "How fast you will drive? predicting speed of customized paths by deep neural network," *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [29] Z. Xing, J. Pei, and E. Keogh, "A brief survey on sequence classification," *ACM Sigkdd Explorations Newsletter*, vol. 12, no. 1, pp. 40–48, 2010.
- [30] M. Dixon, "Sequence classification of the limit order book using recurrent neural networks," *Journal of computational science*, vol. 24, pp. 277–286, 2018.
- [31] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3431–3440, 2015.
- [32] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.
- [33] F. Xia, T.-Y. Liu, J. Wang, W. Zhang, and H. Li, "Listwise approach to learning to rank: theory and algorithm," in *Proceedings of the 25th international conference on Machine learning*, pp. 1192–1199, 2008.
- [34] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask r-cnn," in *Proceedings of the IEEE international conference on computer vision*, pp. 2961–2969, 2017.
- [35] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- [36] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *European conference on computer vision*, pp. 740–755, Springer, 2014.
- [37] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [38] R. Wu and E. J. Keogh, "Fastdtw is approximate and generally slower than the algorithm it approximates," *IEEE Transactions on Knowledge and Data Engineering*, 2020.
- [39] H. I. Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, "Deep learning for time series classification: a review," *Data mining and knowledge discovery*, vol. 33, no. 4, pp. 917–963, 2019.